

MVC and Android

Henry Tang

hktang@ualberta.ca

Department of Computing Science

University of Alberta

CMPUT 301 – Introduction to Software Engineering

Slides adapted from Dr. Hazel Campbell, Dr. Ken Wong, and Dr. Abram Hindle



Model/View/Controller Roles

- Model:
 - Entity layer
 - Complete, self-contained representation of the data managed by the application
 - Provides services to manipulate this data
 - “The back end”
 - Main responsibilities
 - Representation and computation issues
 - Sometimes persistence

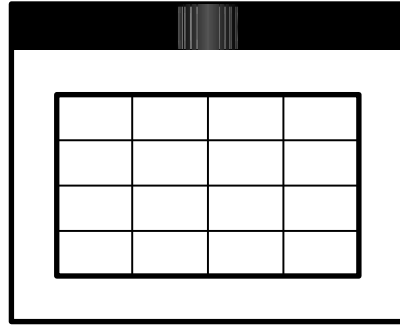
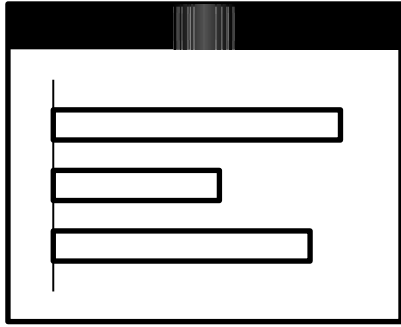
MVC Roles

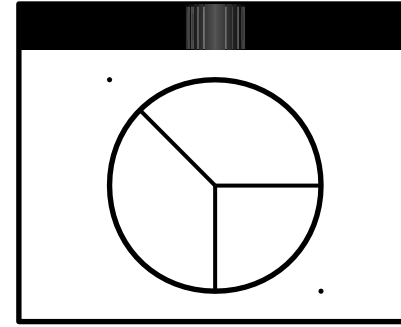
- View:
 - Boundary layer
 - Set of user interface components determines what is needed for a particular perspective of the data
 - “The front end” or what the user sees
 - Main responsibility
 - Display the current data
 - Presentation issues

MVC Roles

- Controller:
 - Control layer
 - Handles events and uses appropriate information from user interface components to modify the model
 - Main responsibility
 - Interaction issues

Views





*Need to maintain
consistency in
the views*

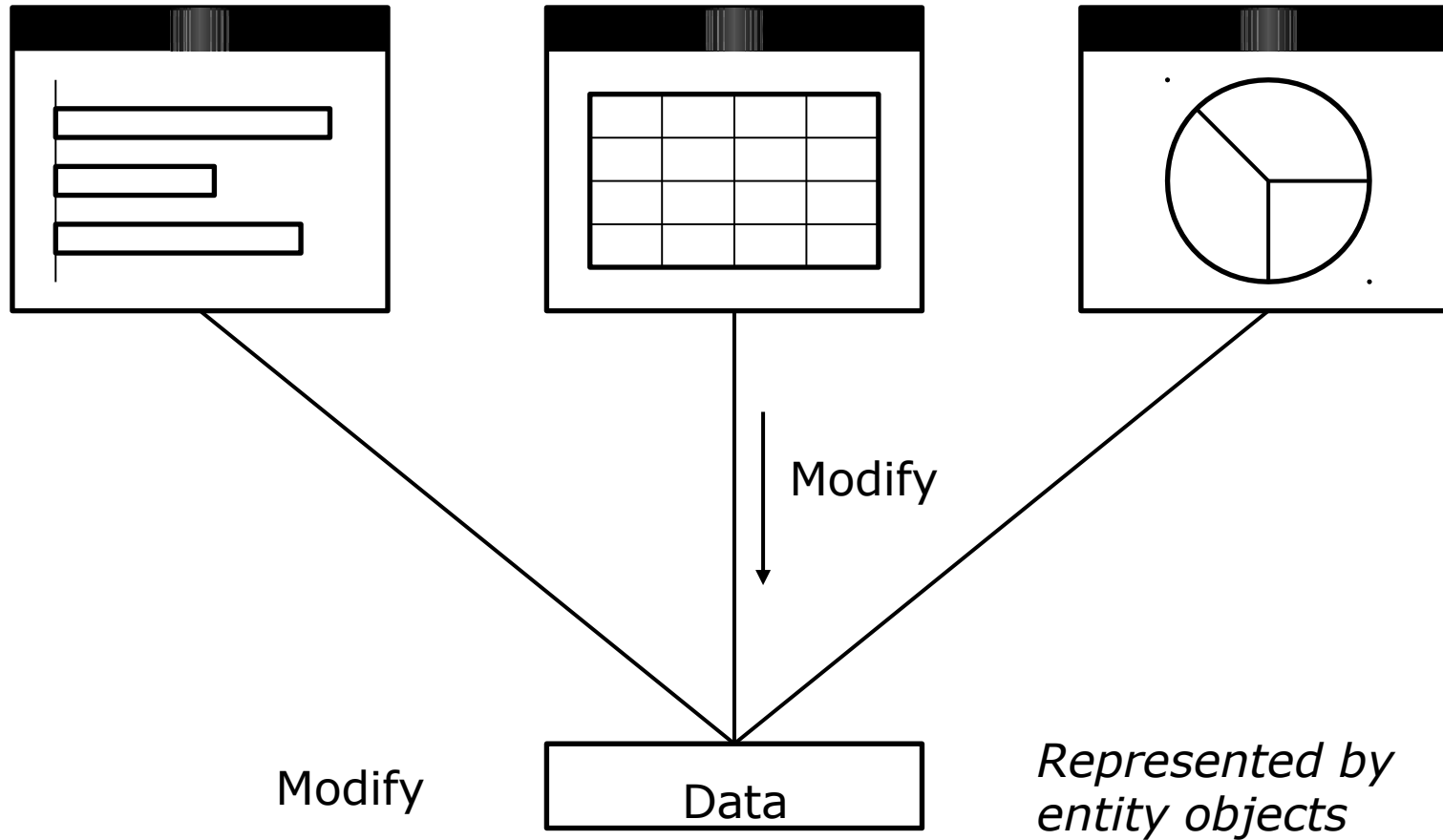
*Want clear, separate responsibilities
for presentation, interaction,
computation, and representation*

Data

*Need to update multiple views
of the common data model*

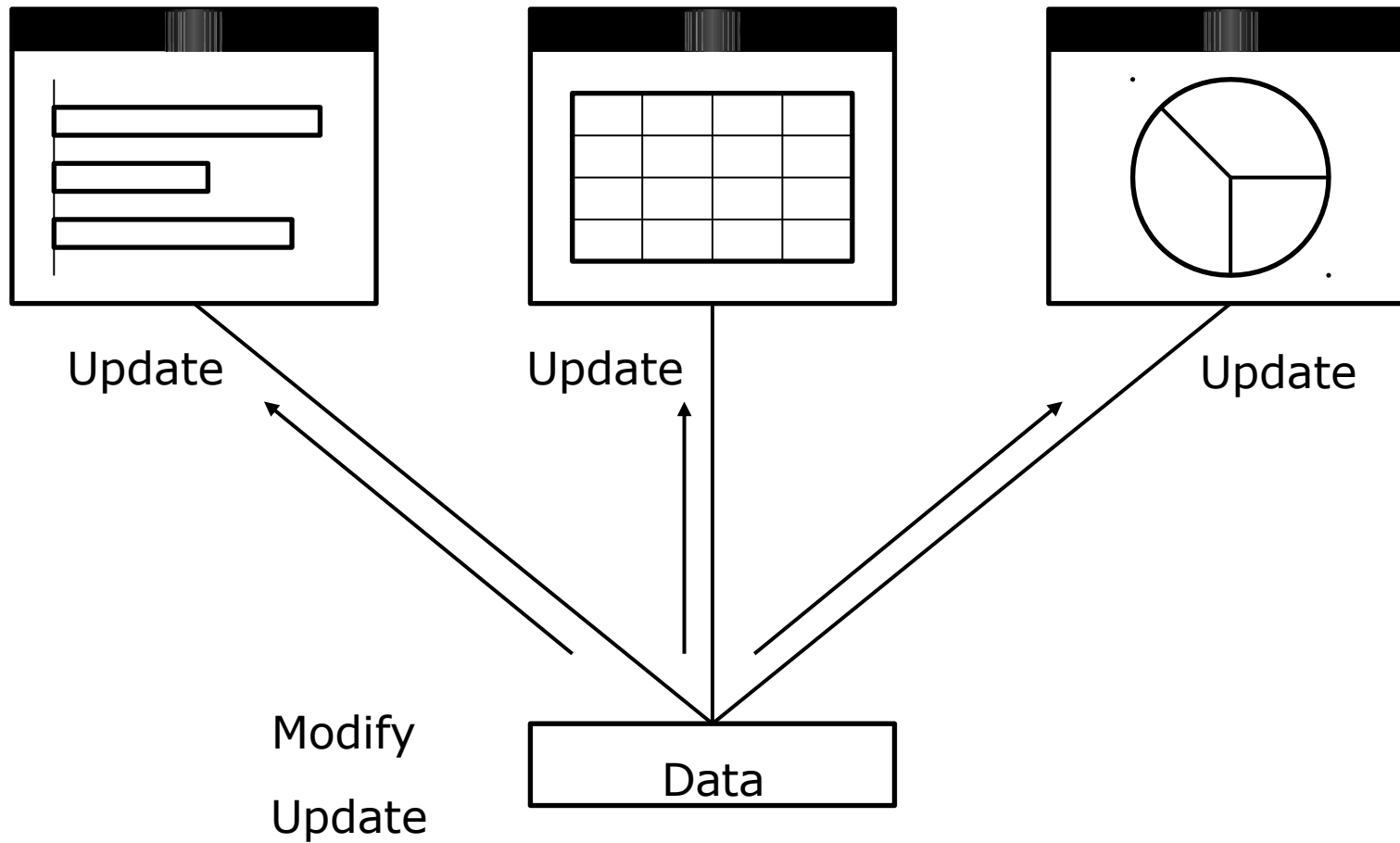
Model

Views (i.e., observers, clients)



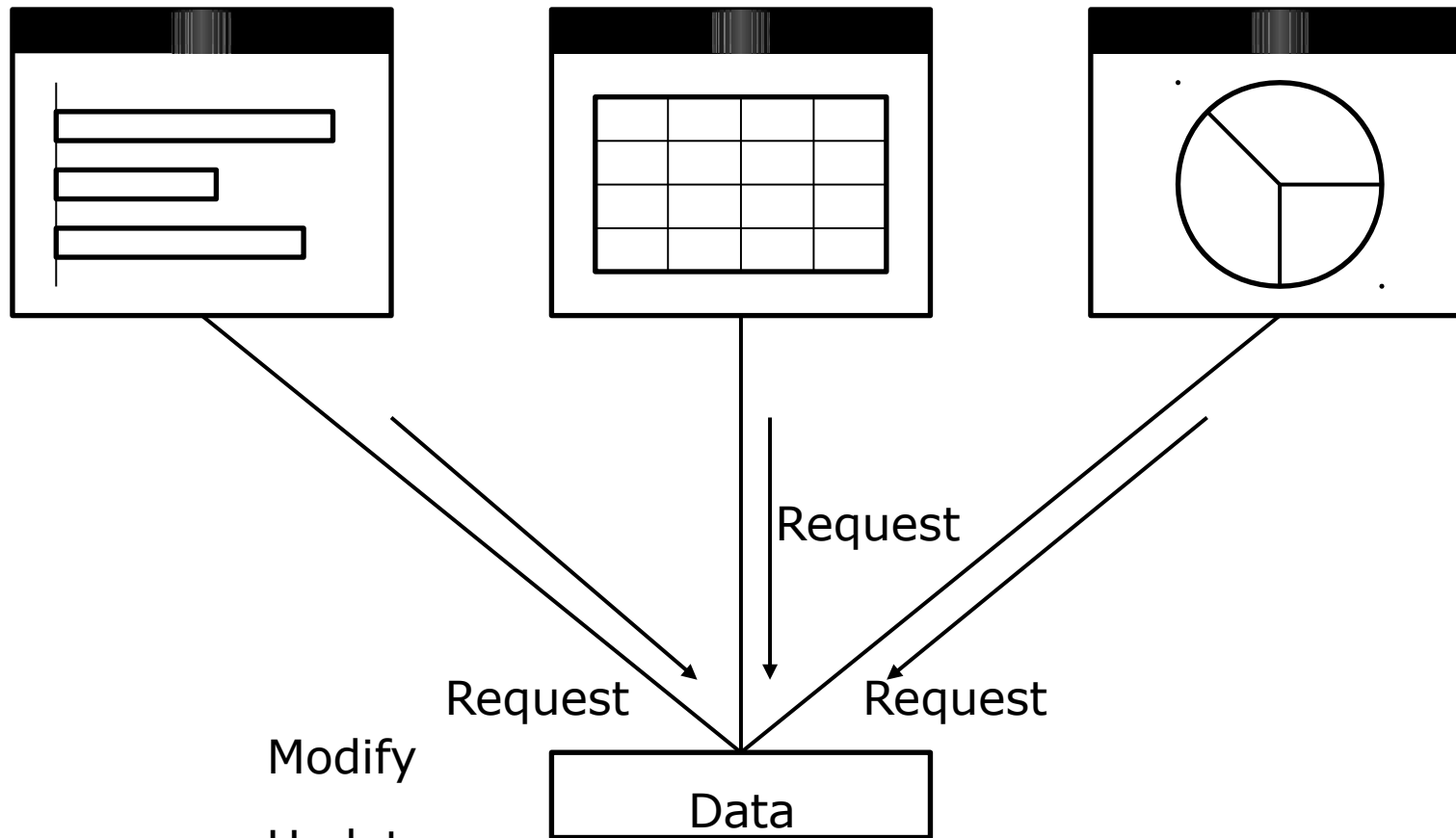
Model (i.e., subject, server)

Views (i.e., observers, clients)



Model (i.e., subject, server)

Views (i.e., observers, clients)



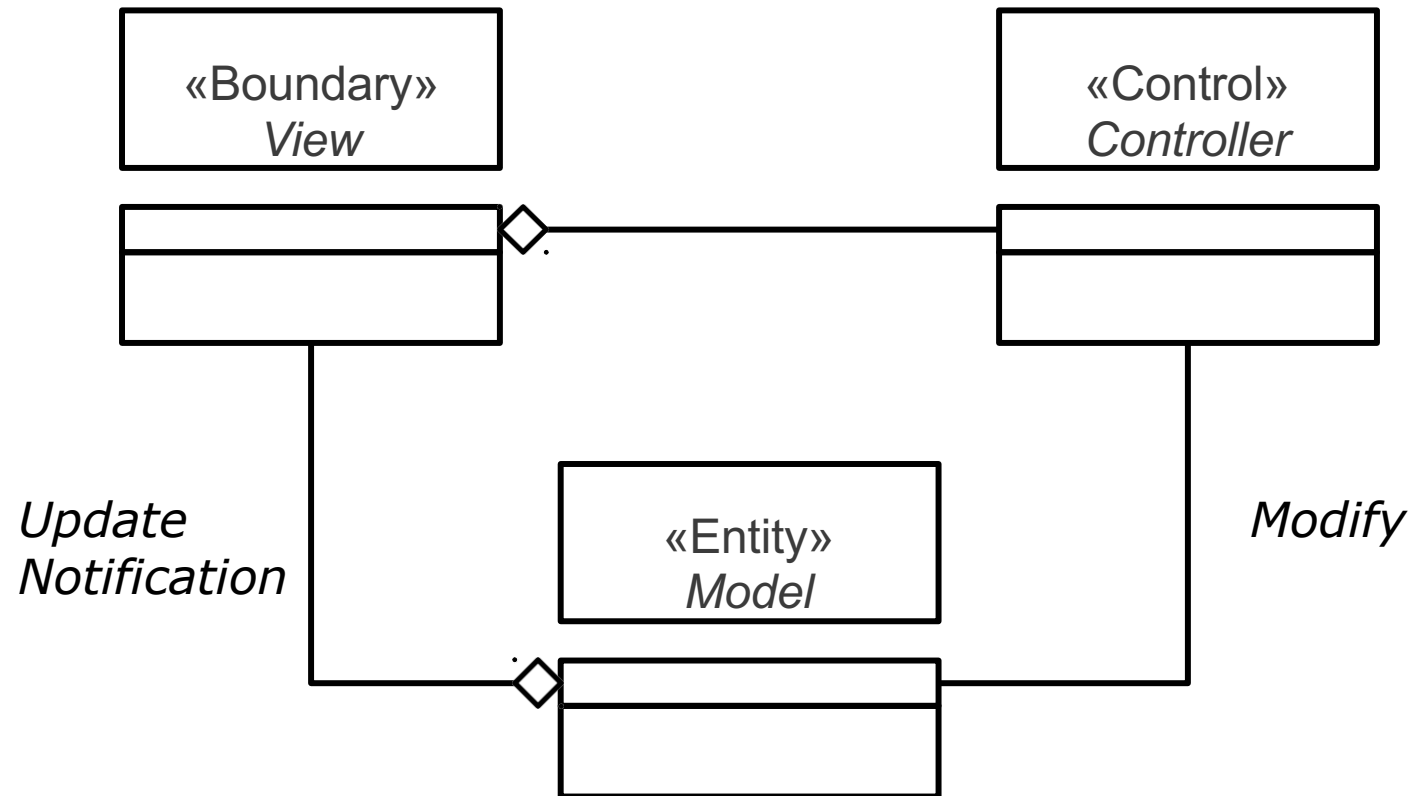
Modify

Update

Request

Model (i.e., subject, server)

*Can create new, specific
types of views without
changing the model*



*The model should not need to know
the particulars of a specific view*

The model should not need
to know about any controllers

MVC Sequence

- User interacts with the View
- Controller handles the interaction
- Controller modifies the Model
- Model changes state
- View updates based on the Model

MVC Design Issues

- Android UI-Dependent part:
 - **Views** contain Android UI components
 - XML layouts, Compose UI, Button, TextView, RecyclerView, etc.
 - **Controllers** handle Android UI events
 - Click listeners, text-change listeners, menu actions, navigation events
- Android UI-Independent part:
 - **Model** should be as Android-free as possible
 - E.g., avoid using Android UI types in entity/data classes
 - Avoid TextView, Button, Context, Activity, Fragment, etc. in Model classes

“MV” Design

- Generalization:
 - Use “model” superclass and “view” interface
 - All models keep track of their views
 - When changed, all models notify their views to update
 - All views update themselves when notified
 - Have application-specific model and view classes

Model-View Update Example

```
interface ViewObserver<M> {  
    fun update( model: M )  
}  
  
open class ObservableModel<M> {  
    private val observers = mutableListOf< ViewObserver<M> >()  
  
    // "all models keep track of their views"  
    fun addObserver( o: ViewObserver<M> ) { ... }  
    fun removeObserver( o: ViewObserver <M> ) { ... }  
  
    // "all models notify their views to update"  
    protected fun notifyObservers( model: M ) { ... }  
  
}
```

Model-View Update Example

```
// MyModel.kt
class MyModel : ObservableModel<MyModel>() {
    var message: String = ""
        private set

    fun updateMessage( newMessage: String ) {
        message = newMessage
        notifyObservers(this) // notify views to update
    }
}
```

Model-View Update Example

```
// MyView.kt
class MyView : ViewObserver<MyModel> {
    override fun update( mode: MyModel ) {
        println( model.message() )
    }
}
```

Model-View Update Example

```
// MyApp.kt
fun main() {
    val theModel = MyModel()
    val aView = MyView()
    val anotherView = MyView()

    theModel.addObserver( aView )
    theModel.addObserver( anotherView )

    theModel.updateMessage( "hello" )
}
```

MVC Design

- Approach:
 - Use a framework that supports MVC to help structure an interactive application
 - Framework is a set of cooperating classes, interfaces, and conventions that forms a reusable design in a particular domain
 - Reusable design *and* code

MVC Framework

Who is in Control?

- Class library reuse
 - Application developers:
 - Write the main body of the application
 - Reuse library code by calling it
- Framework reuse
 - Application developers:
 - Reuse the main body of the application
 - Write code that the framework calls
 - Reuse library code by calling it

Framework

- Separation of concerns:
 - Framework
 - Skeletal application code
 - General superclasses and interfaces
 - Your “customizations”
 - Specific subclasses and implementations
 - Define app-specific models, views, and controllers

Exercise

- Design an MVC framework for building interactive applications.

Generic MVC Structure

```
// MVC.kt
```

```
...
```

```
interface View< M > {  
    fun render( model: M )  
}
```

```
interface Controller {  
    fun onUserAction() // Handles user actions  
}
```

Generic Model

- Generic Model Rules:
 - The Model stores application state
 - The Model contains application rules and calculations
 - The Model should avoid Android UI types
 - The Model may notify Views when its state changes

Generic Model

```
// ObservableModel.kt
```

```
...
```

```
open class ObservableModel<M> {  
    private val views = mutableListOf<View<M>>()  
  
    fun addView( view: View<M> ) {  
        if ( view !in views ) {  
            views.add( view )  
        }  
    }  
}
```

Generic Model

```
fun removeView(view: View<M>) {  
    views.remove(view)  
}
```

```
protected fun notifyViews( model: M ) {  
    views.forEach { view ->  
        view.render( model )  
    }  
}
```

```
}
```

Generic Controller

```
// Controller.kt  
...  
class MyController(  
    private val model: MyModel  
) {  
    fun onUserAction() {  
        // validate input  
        // modify the model  
    }  
}
```

Generic View

```
// MyView.kt
```

```
...
```

```
class MyView : View<MyModel> {  
    override fun render( model : MyModel) {  
        // update the UI using model state  
    }  
}
```

Connecting MVC

```
// MyApp.kt
```

```
...
```

```
val model = MyModel()
```

```
val view = MyView()
```

```
val controller = MyController(model)
```

```
model.addView(view)
```

```
// UI event delegates to the controller
```

```
// e.g., when a button is clicked, call
```

```
    controller.onUserAction()
```

```
}
```

Multiple Views

```
// MyApp.kt
```

```
...
```

```
val model = MyModel()
```

```
model.addView( summaryView )
```

```
model.addView( detailView )
```

```
model.addView( chartView )
```

```
// One Model can update multiple Views
```

```
}
```

Generic MVC Flow

- User action:
 - User interacts with View
 - View delegates interaction to Controller
- Controller response:
 - Controller validates and handles the interaction
 - Controller invokes changes on the Model
- Model update:
 - Model changes state
 - Model notifies registered Views
- View refresh:
 - Views update based on the Model

“Code Reuse”

- [http://www.dilbert.com
/strips/comic/1996-01-31/](http://www.dilbert.com/strips/comic/1996-01-31/)

Example Template

```
class SomeModel : ObservableModel<SomeModel>() {  
    fun changeState() {  
        // update state  
        notifyViews(this)  
    }  
}  
  
class SomeView : View<SomeModel> {  
    override fun render( model: SomeModel ) {  
        // display model state  
    }  
}  
  
class SomeController(  
    private val model: SomeModel  
) {  
    fun onUserAction() {  
        model.changeState()  
    }  
}
```

MVC Responsibility Checklist Example

Question	Belongs in
What data does the application store?	Model
What rules protect or change the data?	Model
What methods change application state?	Model
How is the current state displayed?	View
Which UI components are shown?	View
What happens when the user clicks or types?	Controller
Which Model method should be called for this action?	Controller
Who notifies registered Views after state changes?	Model
Who updates the screen after notification?	View
Who delegates the user interaction to the Controller?	View

Implementing MVC

- Framework:
 - Implement using generic base classes and interfaces for Model and View

TView

```
// TView.kt  
interface TView<M> {  
    fun update( model: M )  
}
```

TModel

```
// TModel.kt
```

```
abstract class TModel<M, V: TView<M> > {  
    private val views = mutableListOf<V>()  
  
    fun addView( view: V ) {  
        if ( view !in views ) {  
            views.add( view )  
        }  
    }  
}
```

```
fun deleteView( view: V ) {  
    views.remove( view )  
}
```

```
protected fun notifyViews(model: M) {  
    views.forEach { view ->  
        view.update( ... )  
    }  
}
```

```
...
```

```
}
```

Example Custom Application

km: 5.00 miles: 3.11

+ 1 km

- 1 km

Custom View

```
// MyLabelView.kt
```

```
...
```

```
class MyLabelView(  
    labelText: String,  
    private val multiplier: Double  
) : TView<MyModel> {  
  
    companion object {  
        private val twoPlaces = DecimalFormat( "0.00" )  
    }  
  
    private val panel = JPanel()  
    private val labelView = JLabel( labelText )  
    private val valueView = JLabel( " " )
```

Custom View

```
init{  
    panel.add( labelView )  
    panel.add( valueView )  
}  
  
val component: JComponent  
    get() = panel  
  
override fun update( model: MyModel ) {  
    val value = model.value * multiplier  
  
    valueView.text = twoPlaces.format( value )  
}  
}
```

Custom Model

```
// MyModel.kt
```

```
class MyModel: TModel< MyModel, TView<MyModel> >() {  
    var value: Int = 0  
        private set  
  
    fun setValue ( newValue: Int ) {  
        value = maxOf( 0, newValue )  
        notifyViews()  
    }  
}
```

Custom Application

```
// MyApp.kt
```

```
...
```

```
class MyApp(  
    title: String,  
    model: MyModel  
) : TApp<MyModel>(title, model) {  
  
    init {  
        // create the UI  
        val myMainView = MyMainView(this, model)  
        // register the main view with the model  
        model.addView(myMainView)  
    }  
}
```

Custom Application

```
fun main() {  
    val model = MyModel()  
    val app = MyApp("MyApp", model)  
  
    model.setValue(0)  
  
    app.content.preferredSize =  
        Dimension(400, 200)  
  
    app.show()  
}
```

Custom User Interface

```
// MyMainView.kt
```

```
...
```

```
class MyMainView(  
    private val app: MyApp,  
    private val model: MyModel  
) : TView<MyModel> {  
    private val kmView: MyLabelView  
    private val milesView: MyLabelView  
  
    private val incrCommand: TCommand  
    private val decrCommand: TCommand  
  
    private val incrButton: JButton  
    private val decrButton: JButton
```

Custom User Interface

```
init{  
    // create views  
    kmView = MyLabelView("km: ", 1.0)  
    milesView = MyLabelView(  
        "miles: ",  
        0.621371192  
    )  
  
    // register views with model  
    model.addView(kmView)  
    model.addView(milesView)
```

Custom User Interface

```
// create commands that modify the model
incrCommand = TCommand {
    model.setValue(model.value + 1)
}
decrCommand = TCommand {
    model.setValue(model.value - 1)
}
```

Custom User Interface

```
// views
app.addToContent( kmView.component )
app.addToContent( milesView.component)

// controls
incrButton = JButton( "+ 1 km" )
app.addToContent( incrButton )

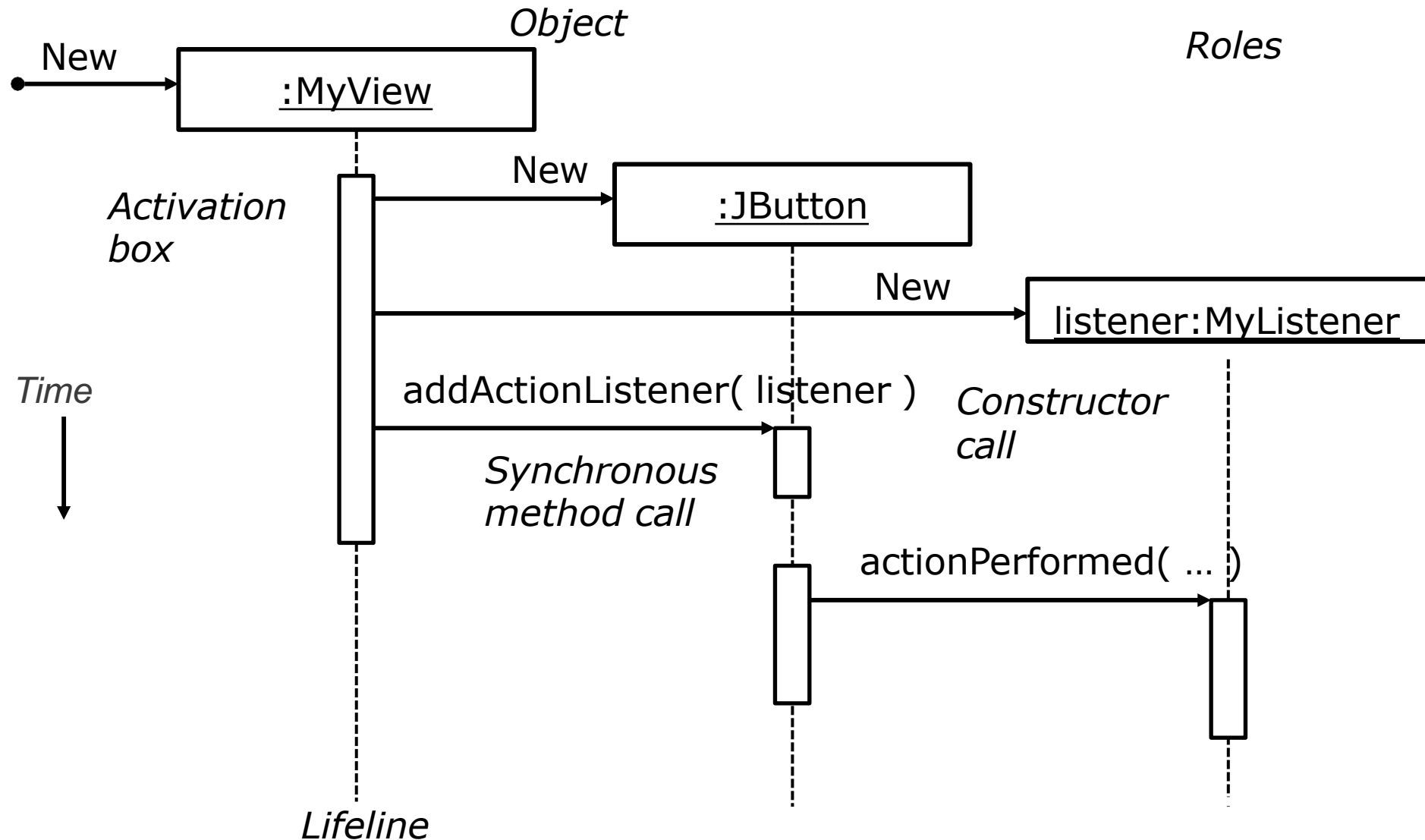
decrButton = JButton( "- 1 km" )
app.addToContent( decrButton )

// connect buttons to commands
TButtonController( incrButton, incrCommand )
TButtonController( decrButton, decrCommand )
}
```

Custom User Interface

```
    override fun update( model: MyModel ) {  
        // nothing to do  
    }  
}
```

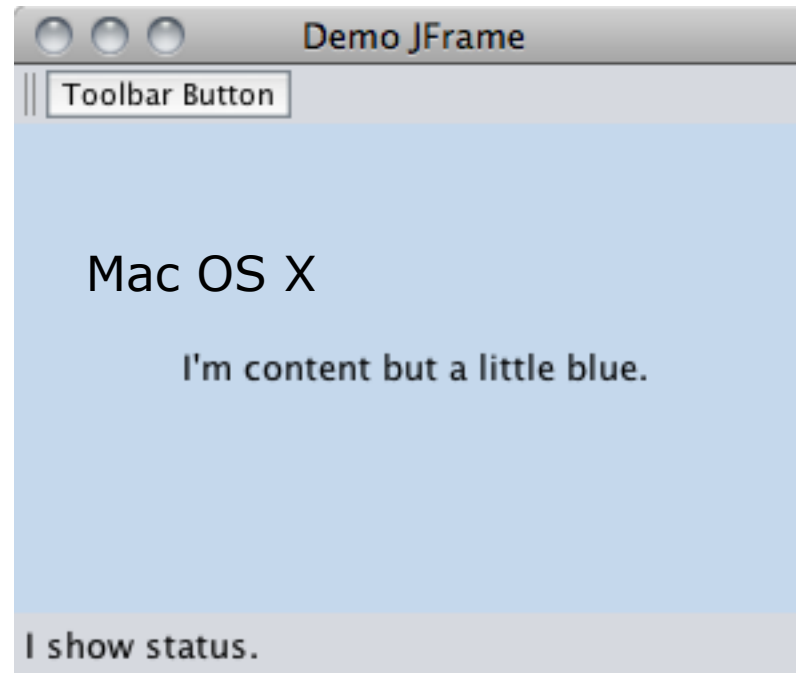
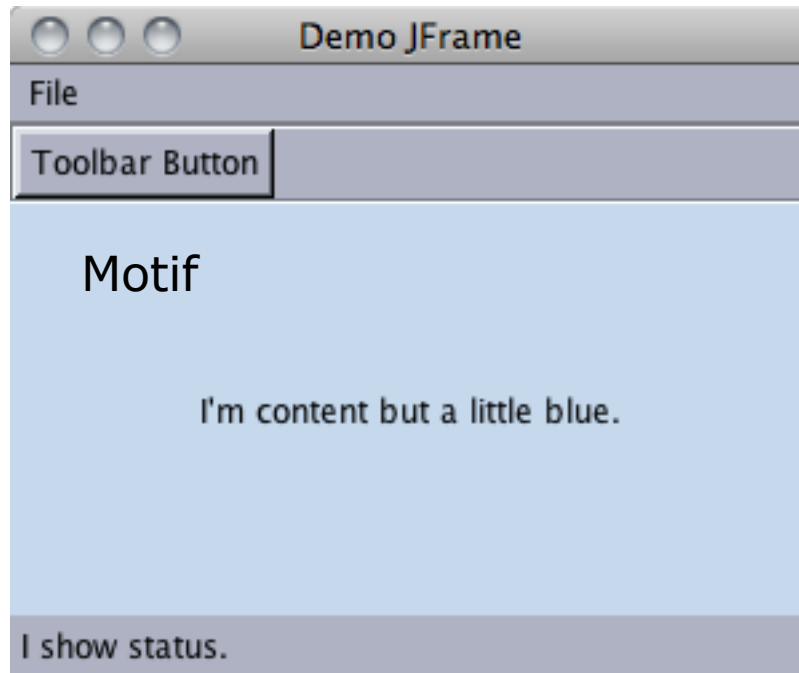
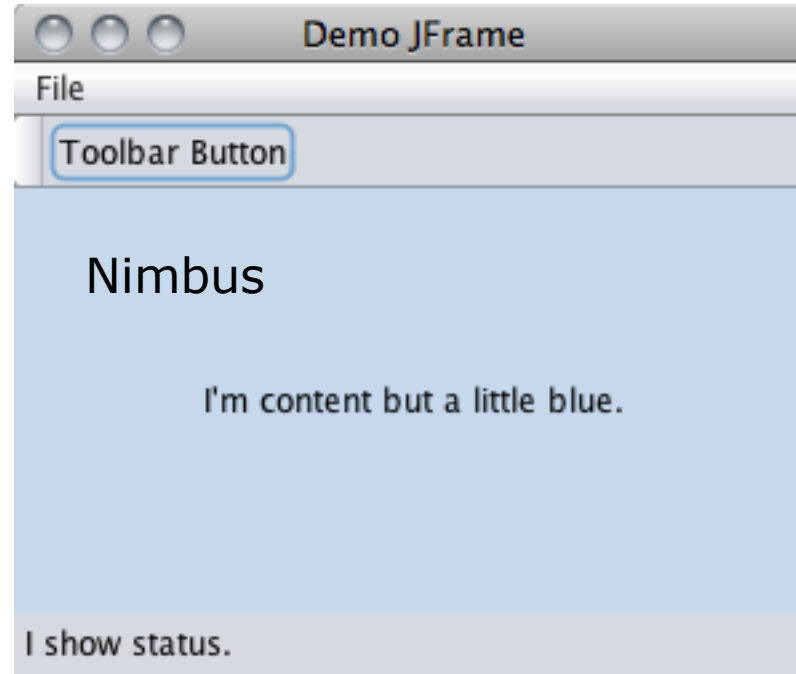
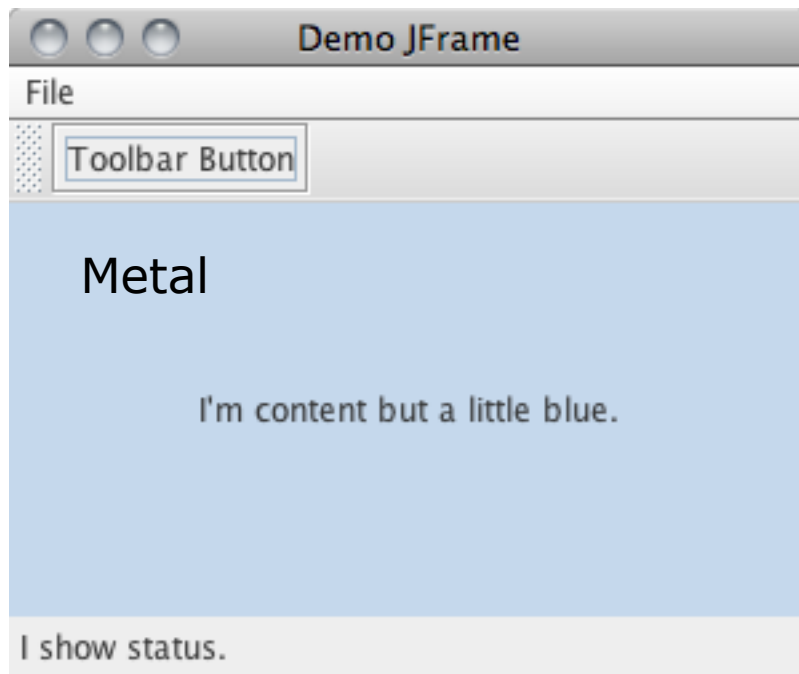
UML Sequence Diagram

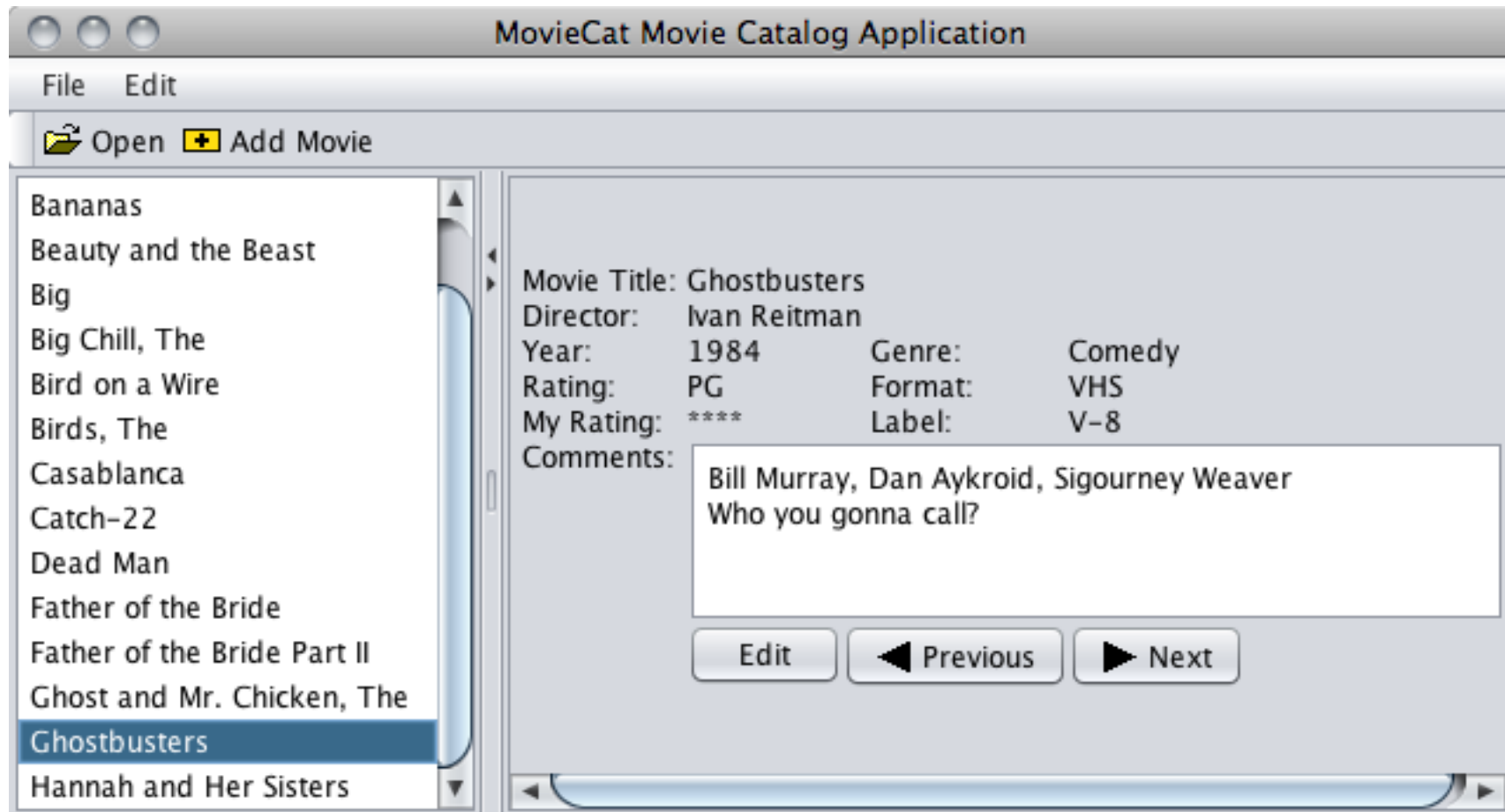


Exercise

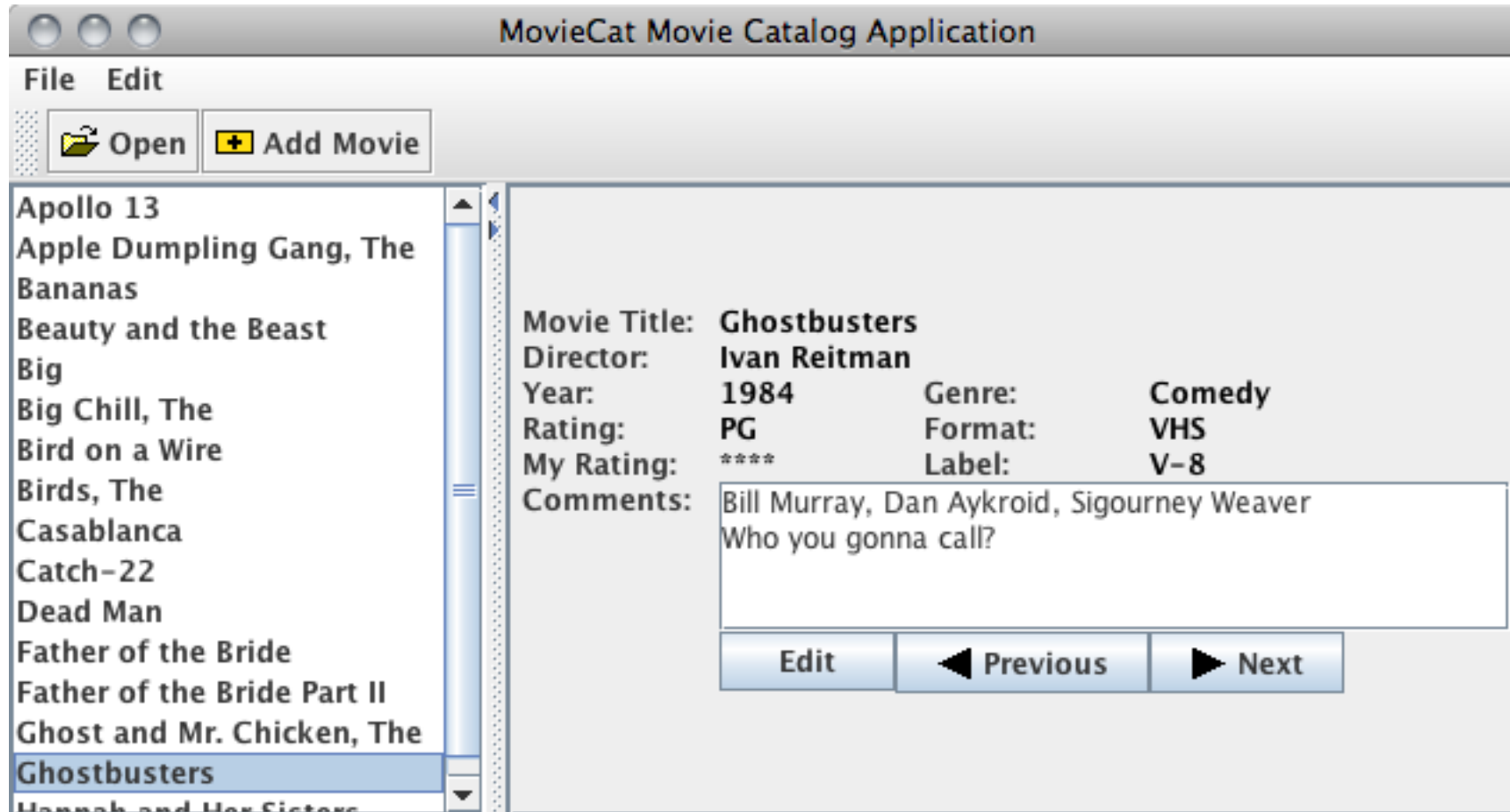
- Draw a UML sequence diagram for the behavior when a button is clicked in the example application.

Swing



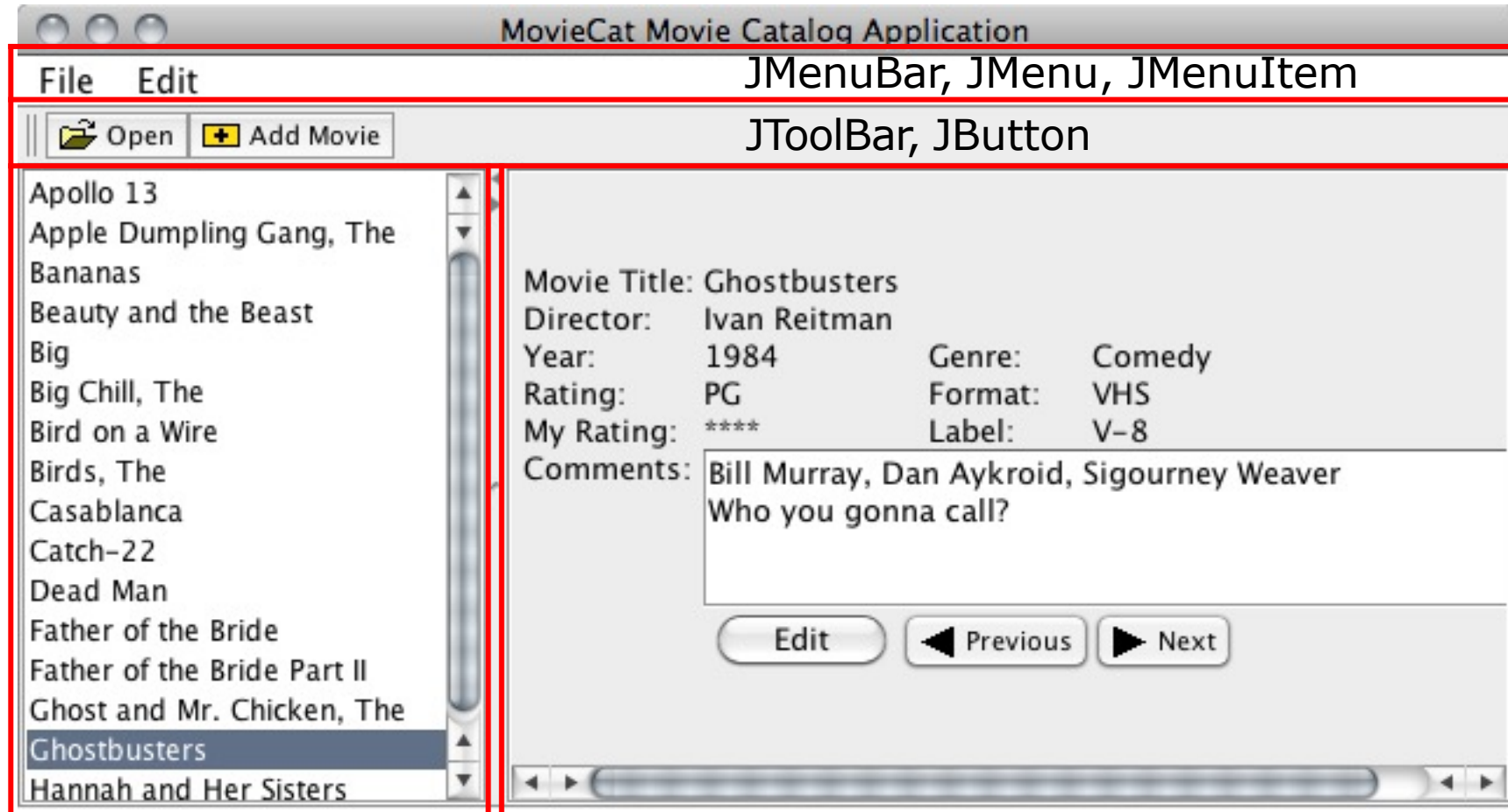


-Dswing.defaultlaf=com.sun.java.swing.plaf.nimbus.NimbusLookAndFeel



-Dswing.defaultlaf=javafx.swing.plaf.metal.MetalLookAndFeel

JFrame, JPanel



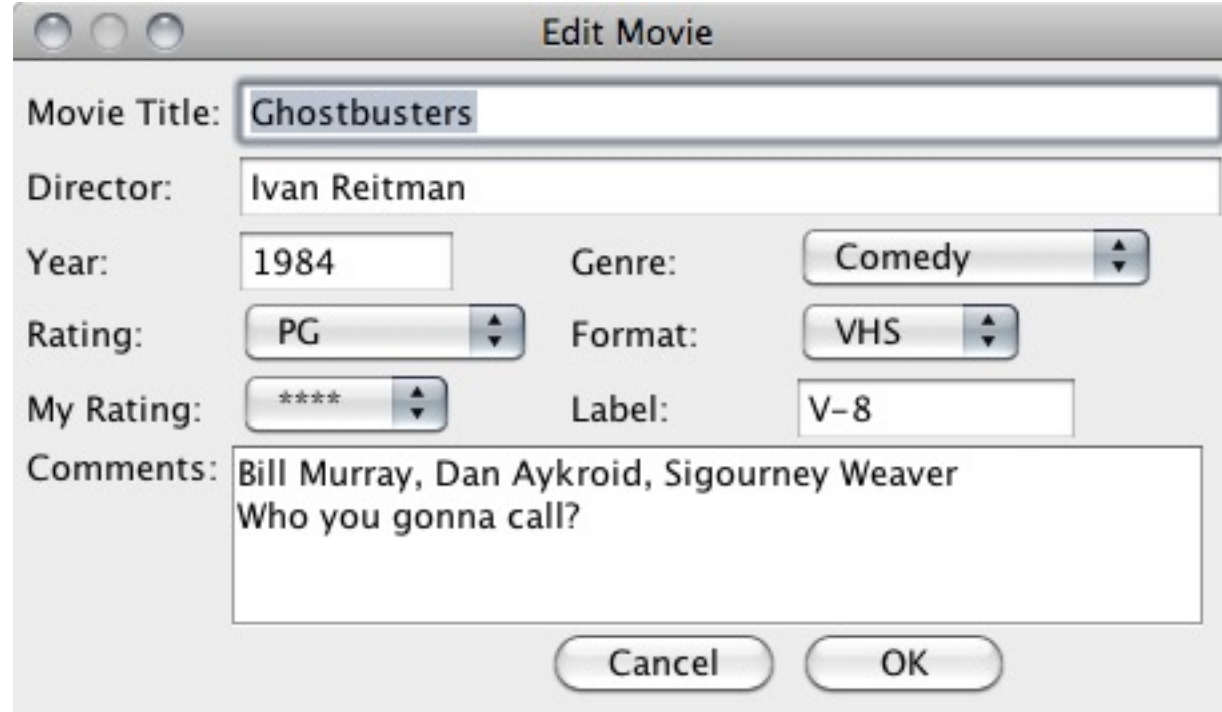
JPanel, JList

JPanel

JSplitPane, JScrollPane

JDialog, JPanel

JLabel



The image shows a Java Swing dialog box titled "Edit Movie". It contains several input fields and buttons. The fields are arranged in a grid-like fashion. The "Movie Title" field contains "Ghostbusters". The "Director" field contains "Ivan Reitman". The "Year" field contains "1984". The "Genre" field is a JComboBox showing "Comedy". The "Rating" field is a JComboBox showing "PG". The "Format" field is a JComboBox showing "VHS". The "My Rating" field is a JComboBox showing "****". The "Label" field contains "V-8". The "Comments" field is a JTextArea containing "Bill Murray, Dan Aykroid, Sigourney Weaver" and "Who you gonna call?". At the bottom of the dialog are two buttons: "Cancel" and "OK".

Movie Title:	Ghostbusters		
Director:	Ivan Reitman		
Year:	1984	Genre:	Comedy
Rating:	PG	Format:	VHS
My Rating:	****	Label:	V-8
Comments:	Bill Murray, Dan Aykroid, Sigourney Weaver Who you gonna call?		
Cancel OK			

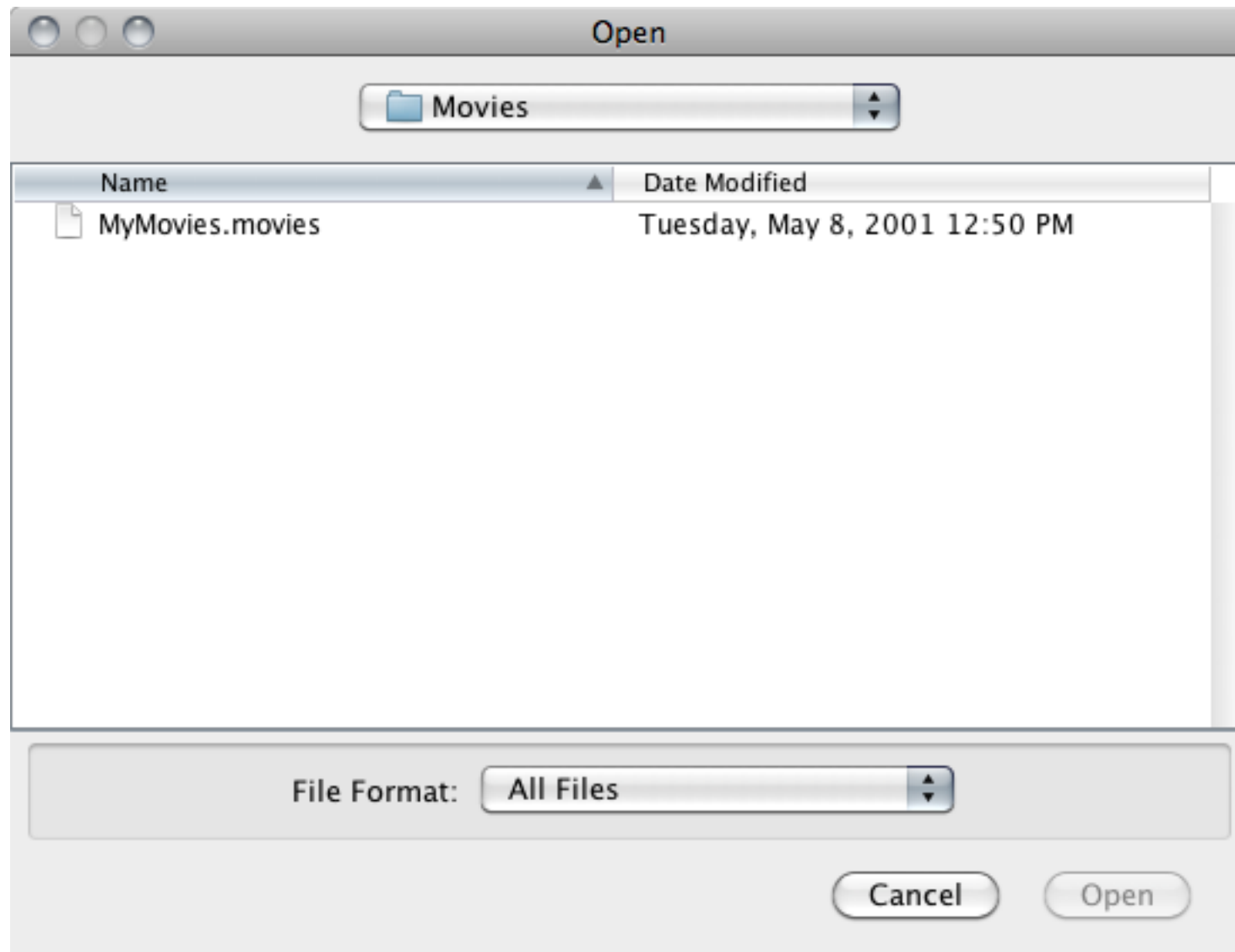
JTextField

JComboBox

JScrollPane,
JTextArea

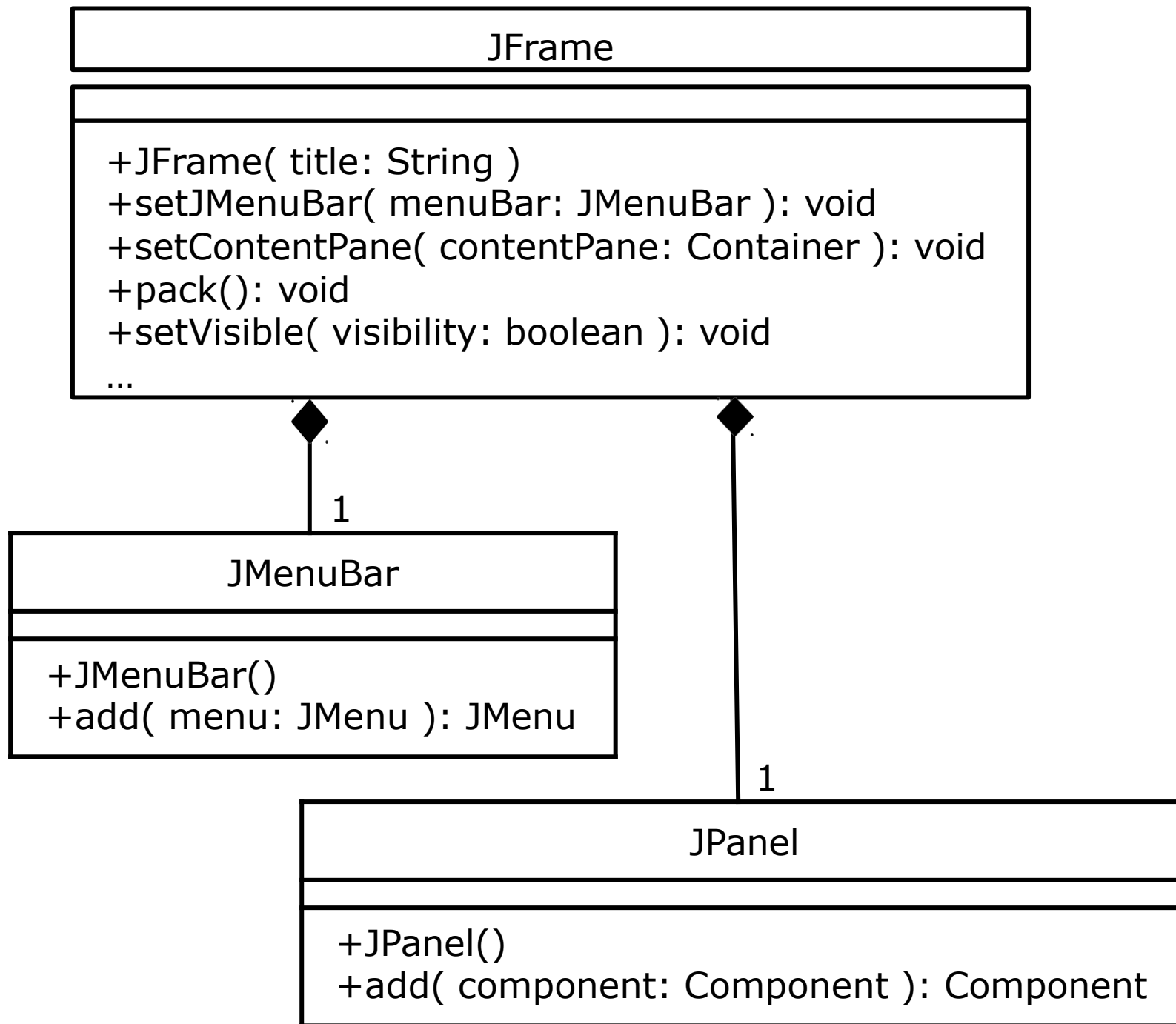
JButton

JFileChooser



Main Window

- Typical containment setup steps:
 - Create a JFrame
 - Create and define a JMenuBar (optional)
 - Add this JMenuBar to the JFrame (optional)
 - Create a JPanel
 - Add components to this JPanel
 - Set JFrame *content pane* to this JPanel
 - Pack and show the JFrame



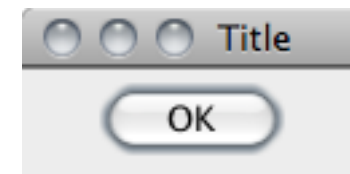
Swing Main Window

```
// MyApp.kt
```

```
import javax.swing.JButton
import javax.swing.JFrame
import javax.swing.JMenuBar
import javax.swing.JPanel

fun main() {
    val frame = JFrame( "Title" )

    val menuBar = JMenuBar()
    // code to define menu items, etc.
    frame.jMenuBar = menuBar
```



```
val panel = JPanel()  
// code to define components in the panel, layout  
// manager, etc.  
val button = JButton( "Hello" )  
panel.add(button)  
  
frame.contentPane = panel  
  
frame.pack()  
frame.isVisible = true  
}
```



Swing Menu Items

```
// code to define menu items, etc.
```

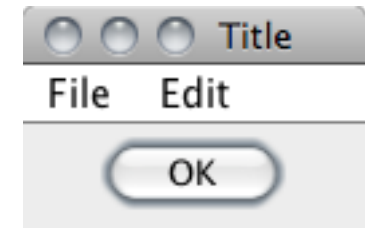
```
val fileMenu = JMenu( "File" )  
val newItem = JMenuItem( "New" )  
val openMenuItem = JMenuItem( "Open" )
```

```
fileMenu.add( newItem )  
fileMenu.add( openMenuItem )  
menuBar.add( fileMenu )
```

```
val editMenu = JMenu( "Edit" )
```

```
...
```

```
menuBar.add( editMenu )
```



Events

- Interactive applications are event driven:
 - Receive an event (e.g., initiated from user)
 - Check event and system state
 - Respond by changing state and display
 - Return and wait for another event
- Event handling is done via:
 - Explicit event loop, event queue, and dispatcher
 - *Registered callback through listeners*

Event Handling

```
// MyListener.kt
```

```
class MyListener : ActionListener {  
    override fun actionPerformed(e:(ActionEvent)) {  
        // react to event  
    }  
}
```

Implementing a Listener

```
// MyView.kt
```

```
class MyListener : ActionListener {  
    override fun actionPerformed( e: ActionEvent ) {  
        // react to event  
    }  
}
```

```
class MyView {  
    init {  
        button.addActionListener( MyListener() )  
    }  
}
```

Implementing a Listener

```
// without adapter class
```

```
class MyWindowHandler : WindowListener {  
    override fun windowClosing(e: WindowEvent) {  
        // respond to closing window  
    }  
  
    override fun windowOpened(e: WindowEvent) {}  
    override fun windowClosed(e: WindowEvent) {}  
    override fun windowIconified(e: WindowEvent) {}  
    override fun windowDeiconified(e: WindowEvent) {}  
    override fun windowActivated(e: WindowEvent) {}  
    override fun windowDeactivated(e: WindowEvent) {}  
}  
  
frame.addWindowListener(MyWindowHandler())
```

More Information

- Books:
 - The Essence of Object-Oriented Programming with Java and UML
 - B. Wampler
 - Addison-Wesley, 2002
 - Java in a Nutshell
 - D. Flanagan
 - O'Reilly, 2005
 - Core Java 2: Fundamentals
 - C. Horstmann
 - Prentice-Hall, 2004

More Information

- Books:
 - Learning Java
 - P. Niemeyer and J. Knudsen
 - O'Reilly, 2005
 - UML Distilled
 - M. Fowler
 - Addison-Wesley, 2003
 - The Elements of UML 2.0 Style
 - S. W. Ambler
 - Cambridge, 2005

More Information

- Links:
 - The Swing Tutorial
 - <https://docs.oracle.com/javase/tutorial/uiswing/>
 - Java Standard Edition 6 API Specification
 - <https://docs.oracle.com/javase/6/docs/api/>
 - Java SE Application Design with MVC
 - <https://www.oracle.com/technical-resources/articles/javase/application-design-with-mvc.html>